WEALTHWISE
UML DESIGN DOCUMENT

Java OOP Wealth Management & Portfolio Simulator
NMIMS • MBA Tech AI • OOP Lab

Document contents
Use case diagram • Class diagram • Buy sequence diagram • Buy activity diagram • Component / deployment view

Design boundary: The browser application and Java console program are separate implementations. The Java program does not serve or synchronize with the web page.

Educational simulation only. Asset names and prices are illustrative; no live market feed or real-money trading is provided.

1. Purpose and Design Boundary

This document provides five UML-style views of WealthWise. The browser simulation demonstrates the user experience; the standalone Java program demonstrates the object-oriented domain model. Diagrams are conceptual and should be read alongside the actual source code.

2. System Overview

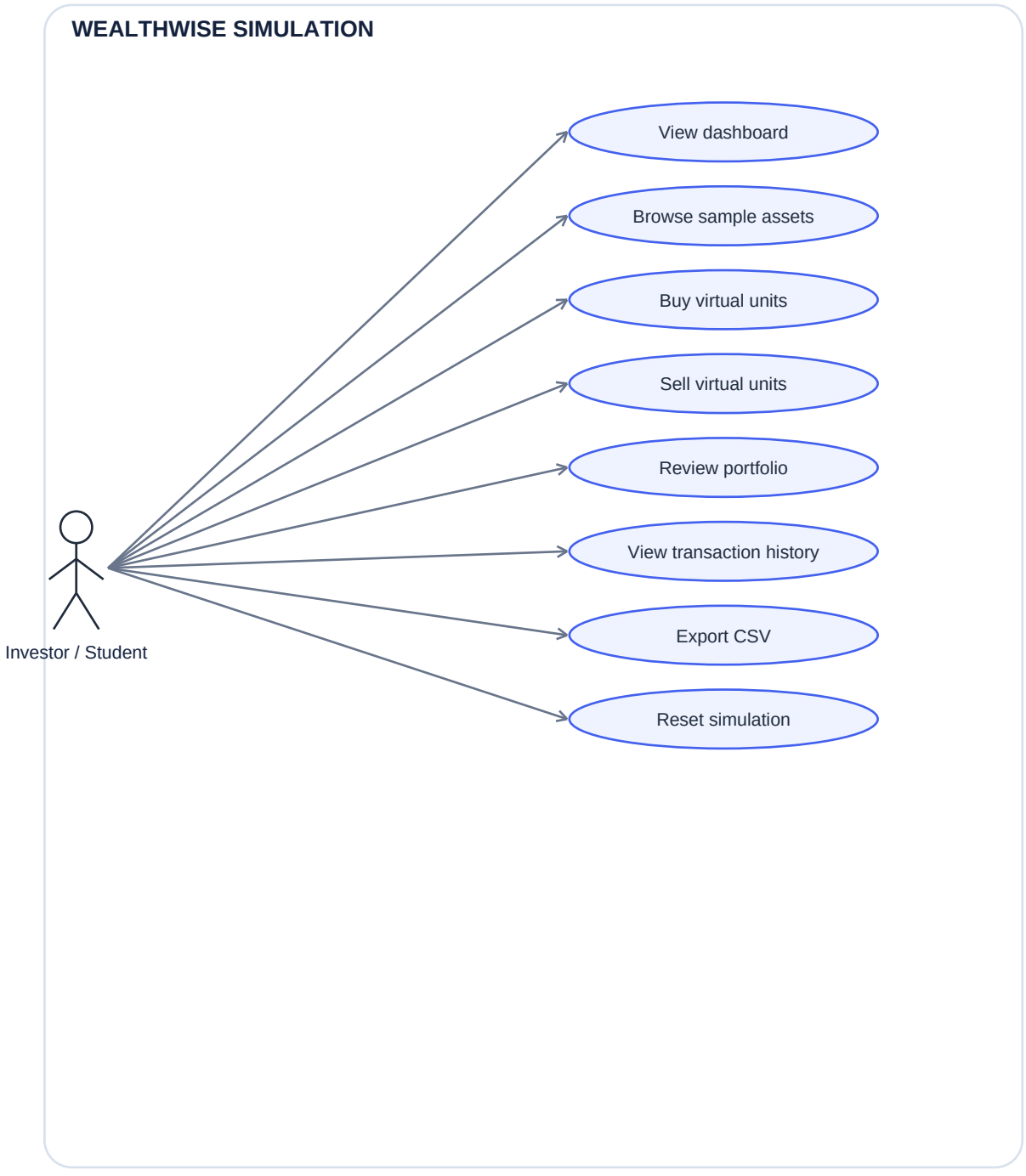
The user can browse sample assets, buy or sell virtual units, review portfolio metrics and activity, export CSV data, and reset the simulation. The Java companion models assets, holdings, portfolio operations, and transaction records. There is no API connection between the two implementations.

3. Diagram Index

Figure 1 — Use Case Diagram
Figure 2 — Class Diagram
Figure 3 — Buy Asset Sequence Diagram
Figure 4 — Buy Operation Activity Diagram
Figure 5 — Component / Deployment View

4. Use Case Diagram

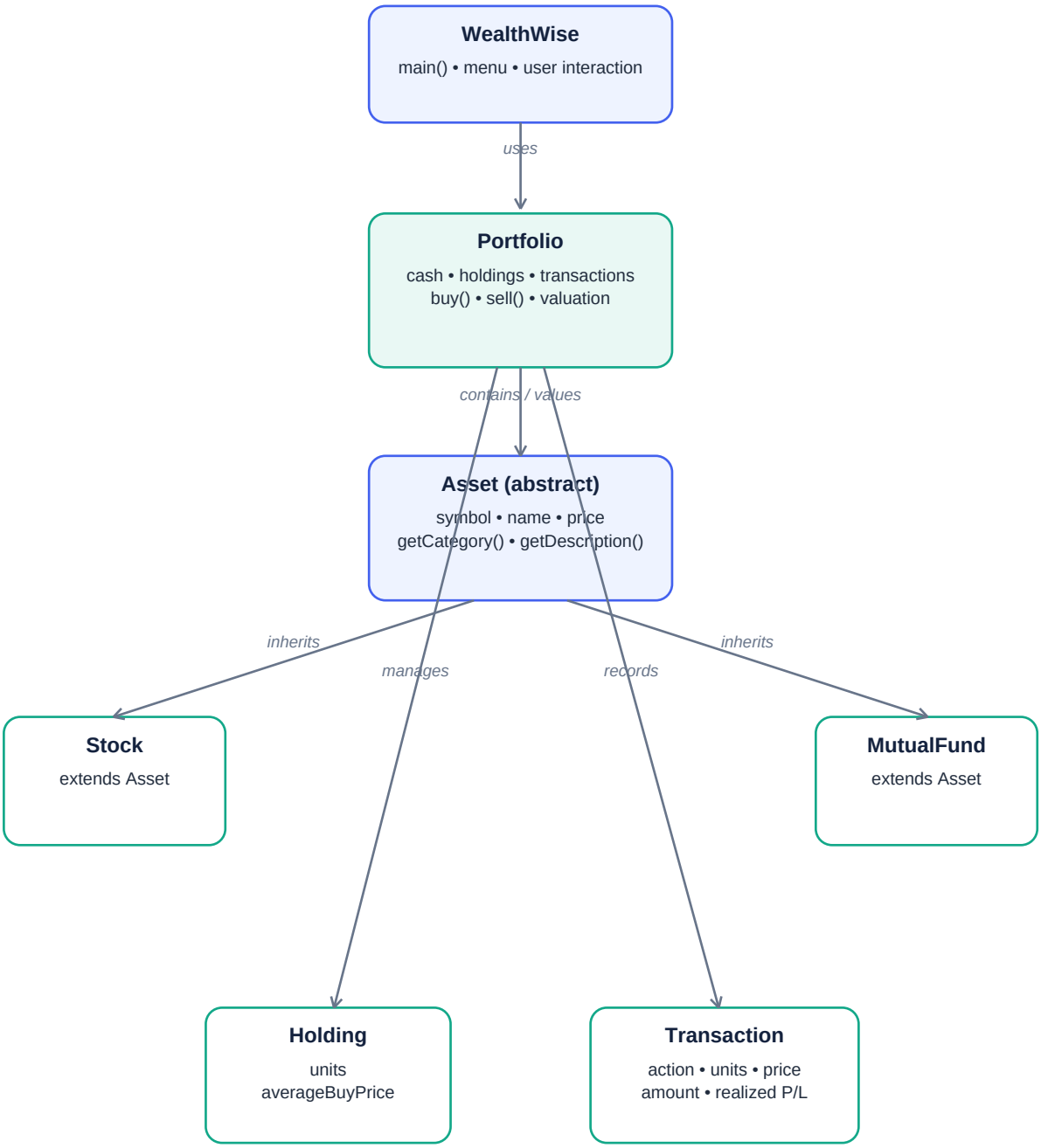
Primary actor: Investor / Student. Use cases represent user-visible goals in the browser simulation.



Use cases describe visible user goals; they are not method-level implementation details.

5. Class Diagram

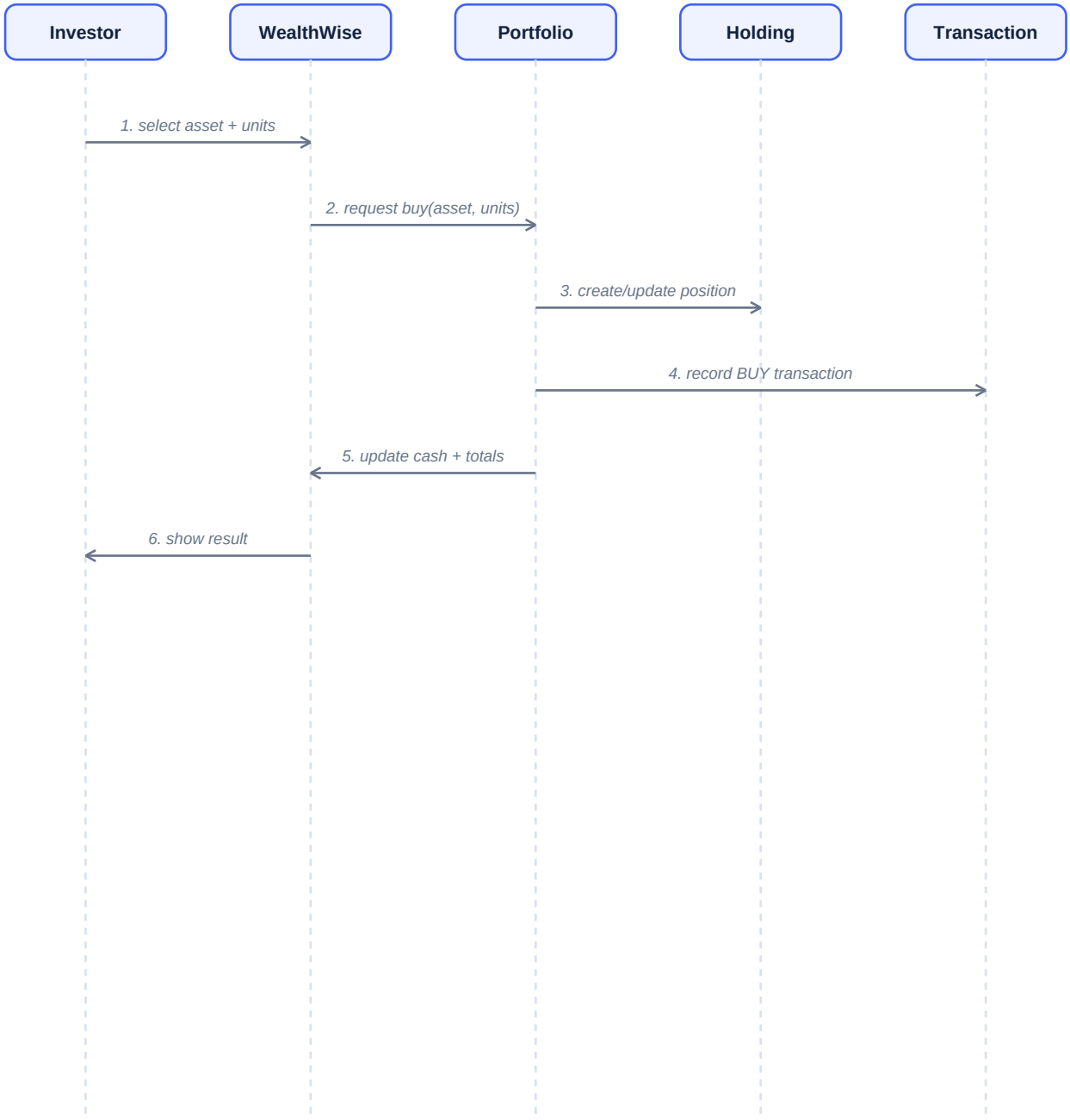
The Java classes below describe the core data model and main relationships in the console companion.



Conceptual class view of the standalone Java companion; nested classes are shown as separate UML boxes.

6. Buy Asset Sequence Diagram

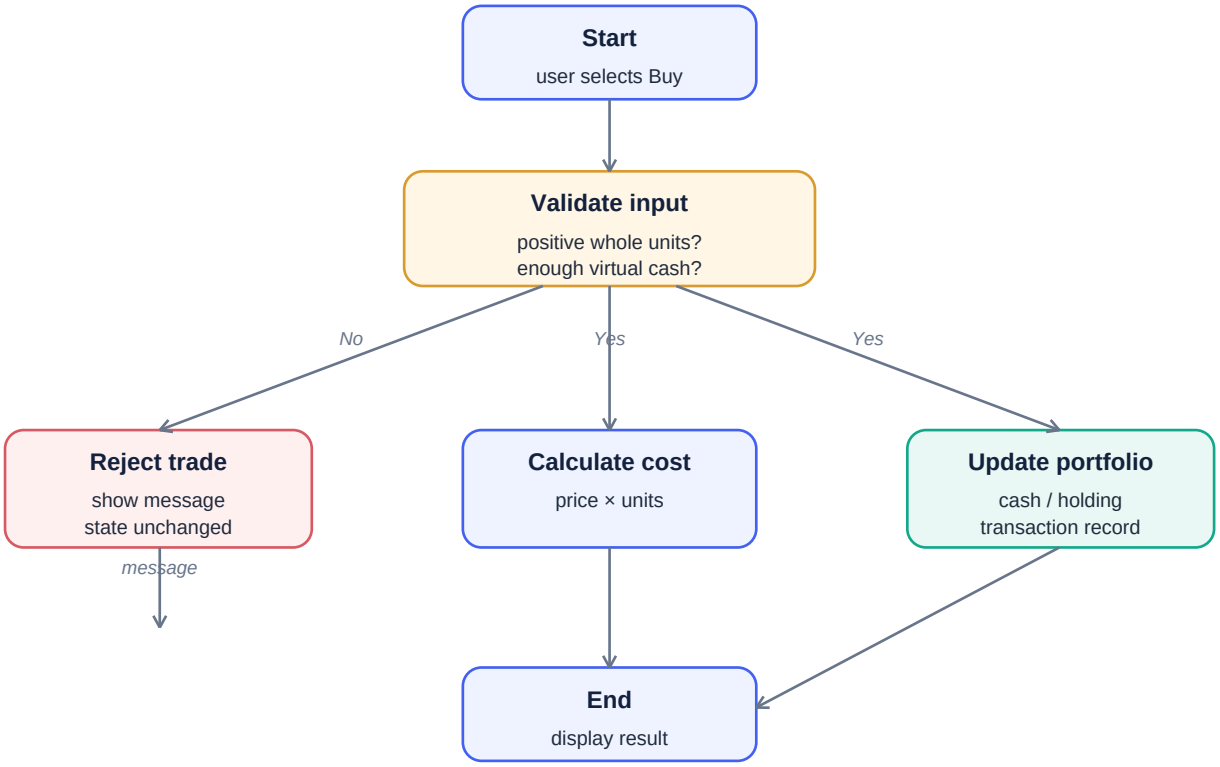
Successful buy path from user selection through validation and portfolio updates.



Illustrates a successful buy path; validation may reject invalid quantities or insufficient virtual cash.

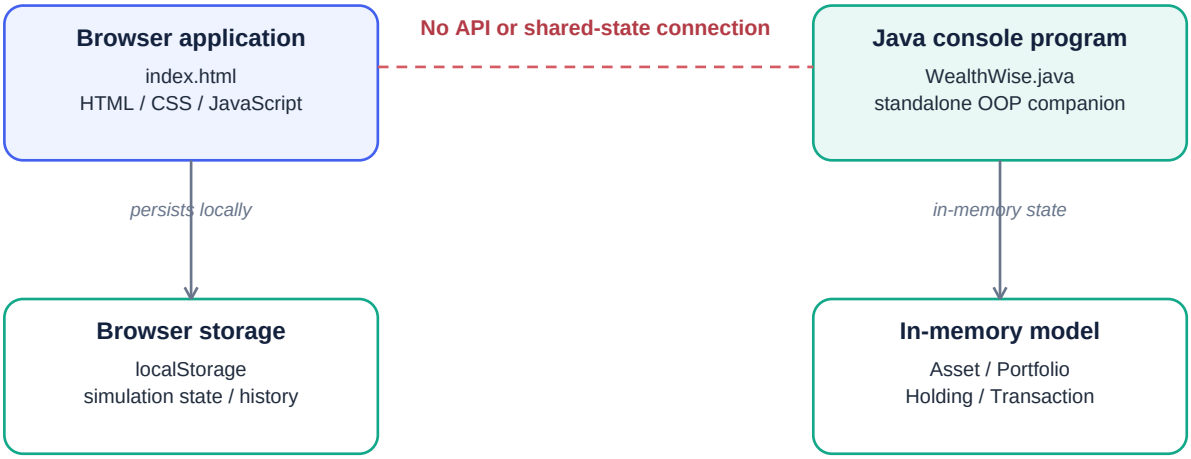
7. Buy Operation Activity Diagram

The operation validates units and available cash before updating virtual portfolio state.



8. Component / Deployment View

Web app stores local state in the browser; Java console has its own in-memory model.



Deployment/component boundary: the web experience and Java program are separate implementations.

9. Class Responsibilities and OOP Mapping

Type	Responsibility
WealthWise	Entry point, user menu, asset selection, dashboard output and simulated market movements.
Asset (abstract)	Common symbol/name/price fields; category and description are abstract operations.
Stock / MutualFund	Concrete asset types overriding category and description.
Holding	Units and average buy price; update or remove units while checking quantities.
Portfolio	Cash, holdings, transactions, buy/sell validation, valuation and profit/loss calculations.
Transaction	Record an action, asset, quantity, unit price, amount, and realized P/L.

Abstraction: abstract Asset defines shared properties and required behaviours. **Inheritance:** Stock and MutualFund extend Asset. **Polymorphism:** overridden methods are called through Asset references. **Encapsulation:** private fields and controlled methods protect portfolio state. **Composition:** Portfolio manages Holding and Transaction objects.

10. Core Rules and Assumptions

• Quantities must be positive whole numbers.
• A buy is rejected when the available virtual cash is insufficient.
• A sale is rejected when the requested units exceed the held units.
• Purchase cost is unit price multiplied by units.
• Unrealized P/L is current holding value minus invested cost; realized P/L is recorded when units are sold.
• Simulated market movements update illustrative asset prices only.
• No live data, broker connection, authentication, or real trading is implemented.

11. Notes for Implementation Review

The UML diagrams are a design aid, not a claim that the web page and Java console application share runtime state. When presenting the project, use the browser application for user-facing behavior and the Java source to explain OOP concepts. Verify the current code and run tests before claiming that any test has passed.